

# Implementation Standard

## CODESYS ↔ RoboDK Interface over OPC UA

### 1. Purpose of This Document

This document describes the complete procedure for installing, configuring and commissioning the interface between:

- CODESYS V3.5 SP20 Patch 1
- RoboDK 5.9
- a Python script integrated in RoboDK
- communication over OPC UA (local endpoint)

The goal is to make the implementation of the system reproducible on a comparable robot station.

### 2. System Architecture

Communication runs through the CODESYS OPC UA server with the following endpoint:

opc.tcp://localhost:4840

Roles

- **CODESYS:** OPC UA server, user interface, generation of the control commands
- **Python script (RoboDK):** OPC UA client, interface logic, IO synchronization
- **RoboDK:** robot execution and program logic

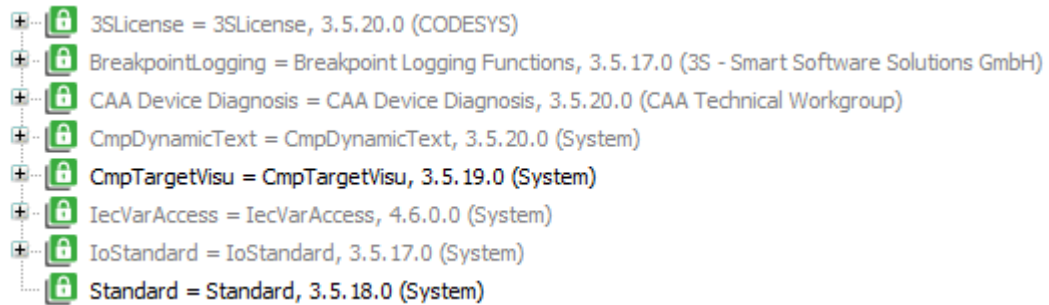
Data flow:

CODESYS → OPC UA → Python → RoboDK  
RoboDK → Python → OPC UA → CODESYS

### 3. Software Requirements

| Software             | Required version            |
|----------------------|-----------------------------|
| <b>CODESYS</b>       | V3.5 SP20 Patch 1           |
| <b>RoboDK</b>        | 5.9                         |
| <b>Python script</b> | Single script inside RoboDK |

Installed CODESYS libraries:



## 4. Security Settings in CODESYS

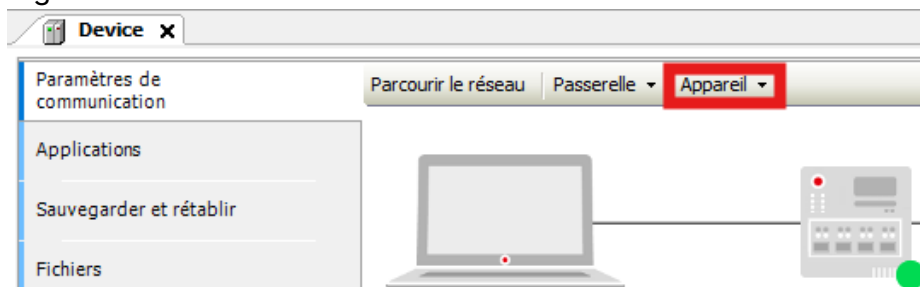
The security policy of the device must be adjusted before the first commissioning.

Procedure:

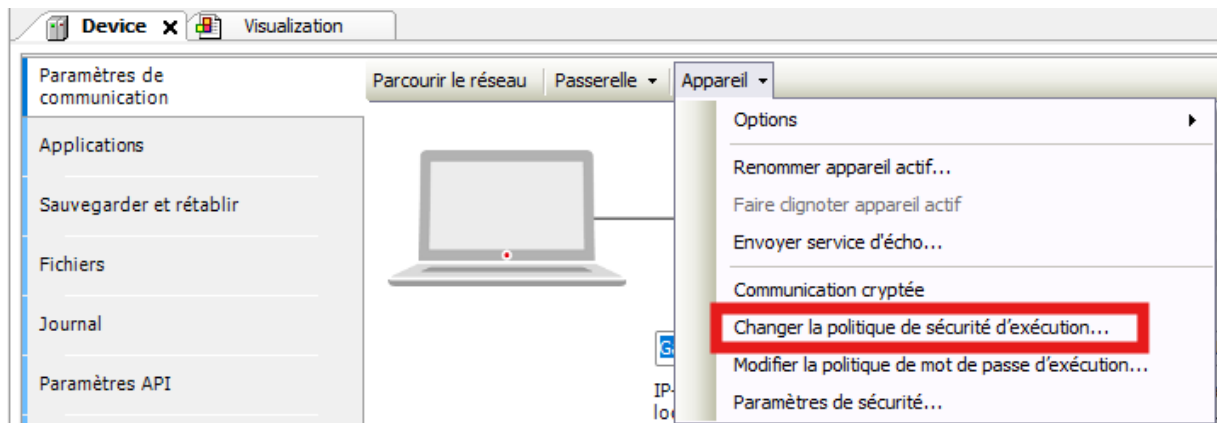
1. Open the **Device** tab



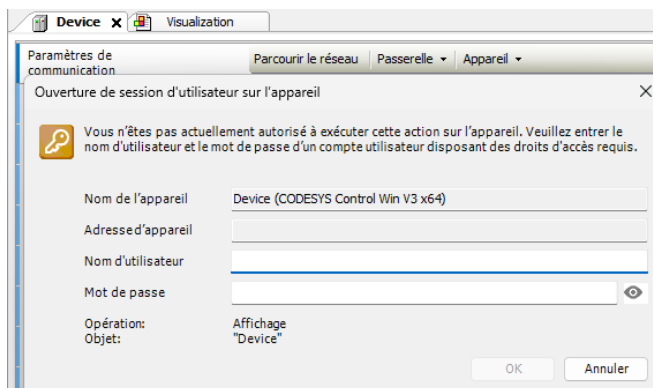
2. Right-click on **the device**



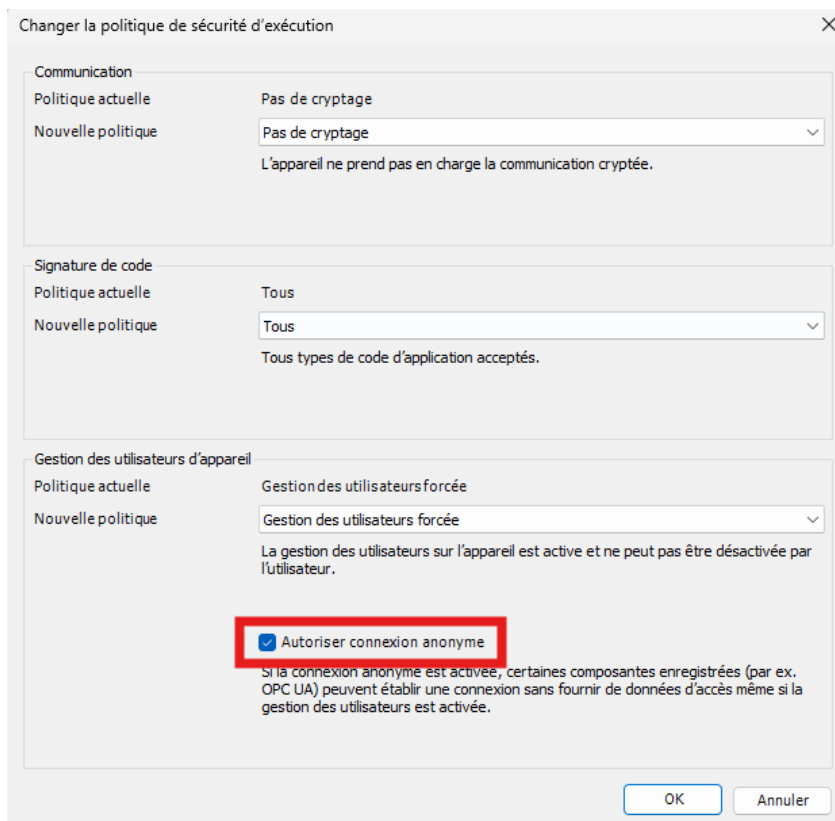
3. Select "Change Execution Security Policy"



4. Enter user name and password (if the connection has not been established yet)



5. Enable "Allow anonymous connection"



This setting only needs to be made once and stays saved after CODESYS is closed.

Without this adjustment, no OPC UA connection can be established.

## 5. Configuring the RoboDK Station

### 5.1 Defining the Controllable Programs

#### 5.1.1 Naming Convention

Programs that are to be controlled from the CODESYS interface:

- must contain **no spaces** in the name
- must be **uniquely named**

Allowed:

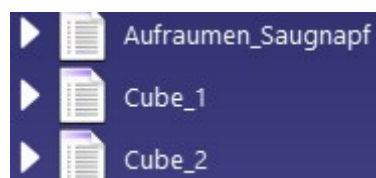
Cube1  
WeldA  
ProgramTest

Not allowed:

Cube 1  
Weld A

Spaces cause misinterpretation in the Python script.

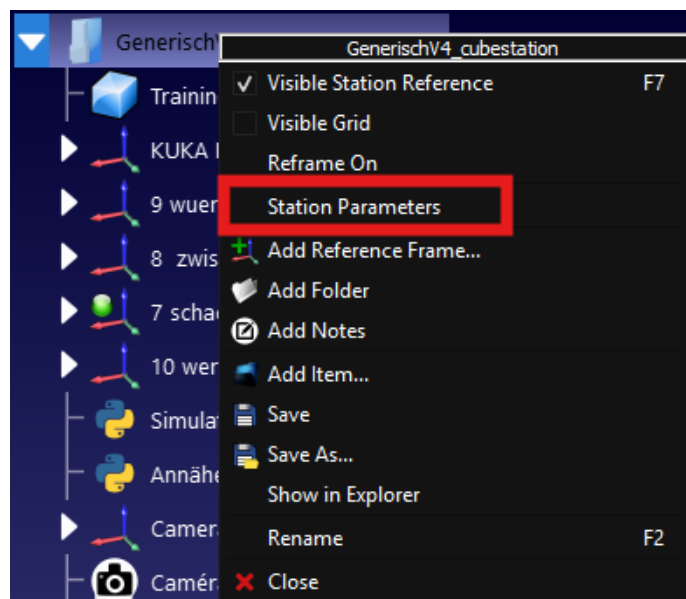
E.g.:



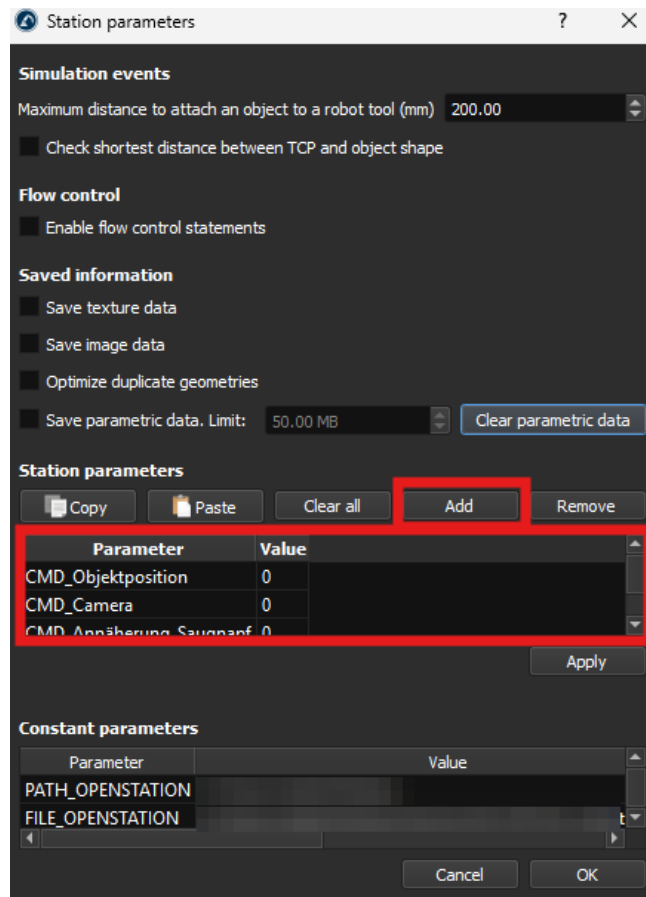
### 5.1.2 Entry in the Station Parameters

Procedure:

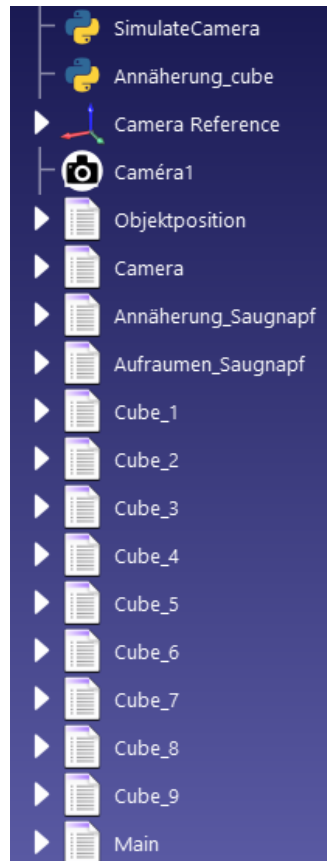
1. Right-click on the station name



2. Open "Station parameters"



3. For each controllable program, add a parameter in the following form:



| Parameter               | Value |
|-------------------------|-------|
| CMD_Objektposition      | 0     |
| CMD_Camera              | 0     |
| CMD_Annäherung_Saugnapf | 0     |
| CMD_Aufräumen_Saugnapf  | 0     |
| CMD_Cube_1              | 0     |
| CMD_Cube_2              | 0     |
| CMD_Cube_3              | 0     |
| CMD_Cube_4              | 0     |
| CMD_Cube_5              | 0     |
| CMD_Cube_6              | 0     |
| CMD_Cube_7              | 0     |
| CMD_Cube_8              | 0     |
| CMD_Cube_9              | 0     |
| CMD_Main                | 0     |
| IO_Run                  | 0     |
| CMD_SimulateCamera      | 0     |
| CMD_Annäherung_cube     | 0     |

CMD\_ProgramName

Example:

Program name:

Cube\_1

Entry in the station parameters:

CMD\_Cube\_1

The CMD\_ prefix lets the Python script automatically detect all controllable programs.

## 5.2 Defining the IO\_Run Variable

The following variable must also be created in the station parameters:

IO\_Run → 

|        |   |
|--------|---|
| IO_Run | 0 |
|--------|---|

This variable signals that a program is currently running and is used for synchronization with the CODESYS interface.

## 6. Structure of the RoboDK Programs

Each controllable program is assigned to a specific IO signal (IO\_n).

Two program types are distinguished:

1. Built-in RoboDK programs
2. Python-based RoboDK programs

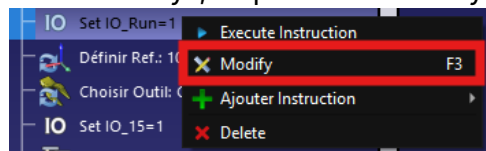
### 6.1 Built-in RoboDK Programs

IO instructions must be added to each program. Here is how:

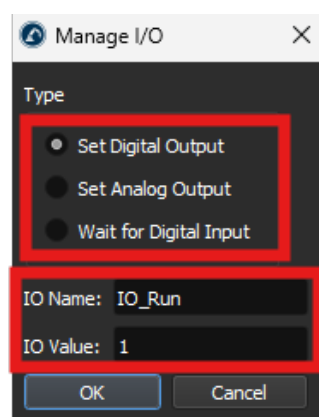
1. Find the icon in the toolbar



2. Right-click it and select "Modify", or press F3 directly.



3. Choose the type of action (Set IO or Wait IO) and enter the name and the value of the IO.



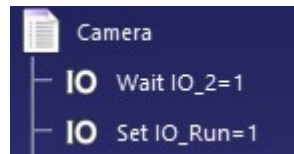
If you choose the type "Set" with a value of 1, the instruction sets the IO to 1. If you choose the type "Wait" with a value of 1, the instruction waits until the IO is at 1 before the program continues.

### 6.1.1 Mandatory Basic Structure

First line of the program:

Wait IO\_n = 1

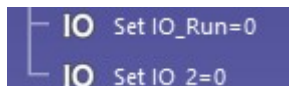
Set IO\_Run = 1



Last lines of the program:

Set IO\_Run = 0

Set IO\_n = 0



The order must be followed strictly.  
IO\_Run must be reset first, then IO\_n.

### 6.1.2 Calling Controllable Subprograms

If another controllable subprogram is called inside a program, it must not be modified directly.

The IO is activated in the parent program, immediately before the call.

Example:

Set IO\_3 = 1

Call Annäherung\_Saugnapf

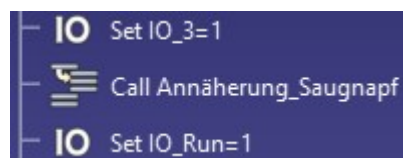
Here, IO\_3 is assigned to the subprogram Annäherung\_Saugnapf.

Optionally, the following addition can be made for visual stabilization:

Set IO\_3 = 1

Call Annäherung\_Saugnapf

Set IO\_Run = 1



With Python subprograms, however, a slight delay can occur. As a result, the IO\_Run light switches off for a brief moment.

### 6.1.3 Internal Functions

For internal RoboDK functions in which no IO commands can be inserted, the program start and end are simulated in the main program.



Example:

Set IO\_n = 1

Call Weld\_A

Set IO\_n = 0



An additional Set IO\_Run = 1 is not required in this case.

## 6.2 Example of a Compliant Built-in Program



## 6.3 Python-Based RoboDK Programs

### Program start:

Simulation of the instruction Wait IO\_n = 1:

```
RDK = Robolink()
```

```
while RDK.getParam("IO_n") != "1":
```

```
    time.sleep(0.05)
```

```
RDK.setParam("IO_Run", "1")
```

```
1  from robolink import *
2  from robodk import *
3
4  RDK = Robolink()
5  while RDK.getParam("IO_15") !=1:
6      time.sleep(0.05)
7  RDK.setParam("IO_Run", "1")
```

### Program end:

```
RDK.setParam("IO_Run", "0")
```

```
RDK.setParam("running", "0")
```

```
RDK.setParam("IO_n", "0")
```

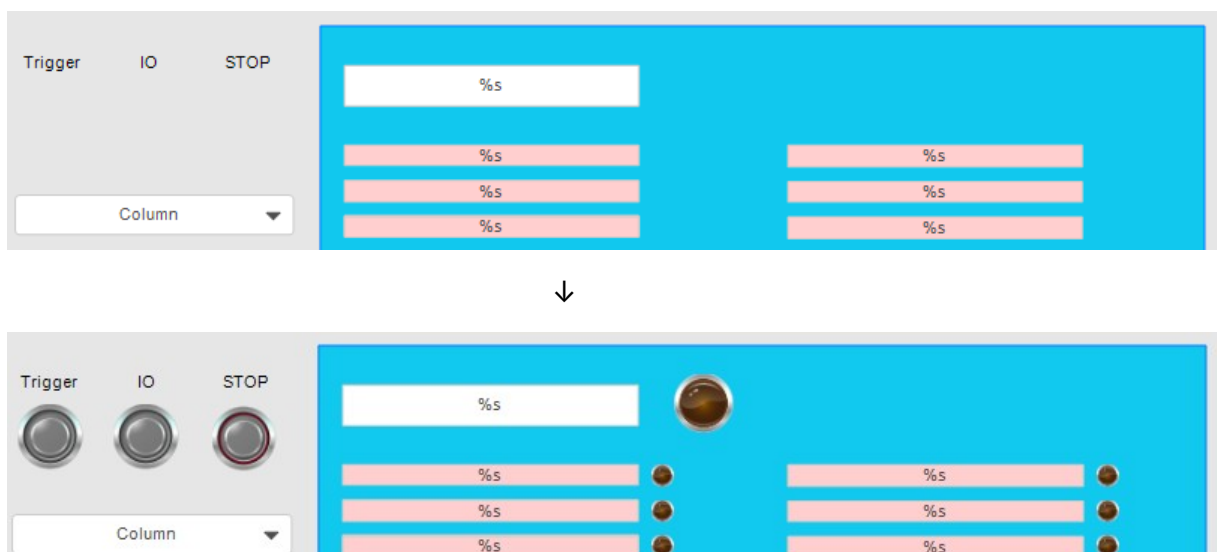
The order must be followed.

```
77 RDK.setParam("IO_Run", "0")
78 RDK.setParam("running", "0")
79 RDK.setParam("IO_15", "0")
```

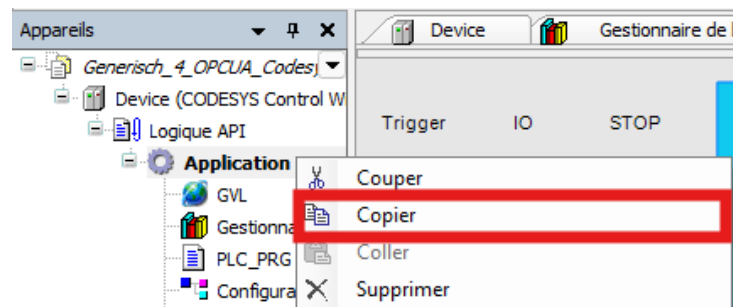
## 7. Commissioning

Commissioning sequence:

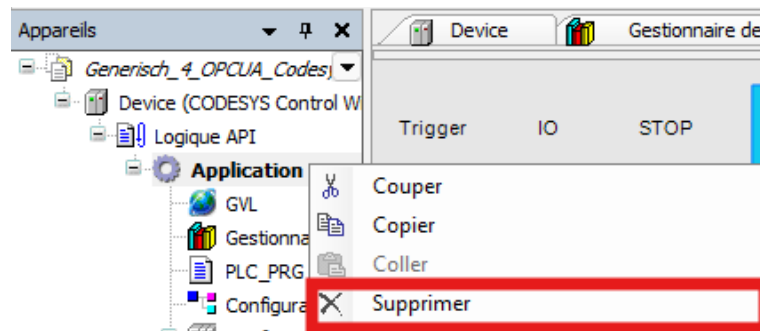
1. Open the RoboDK station
2. Open the CODESYS project
3. If necessary, fix visualization errors (if control elements and lamp elements are not displayed):



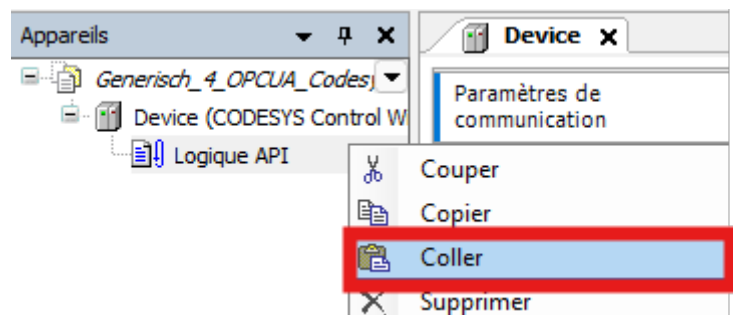
- o Copy the "Application" folder



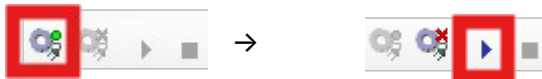
- o Delete the original



- o Paste it back into "Logik API"



#### 4. Start the CODESYS runtime



#### 5. Start the Python script in RoboDK

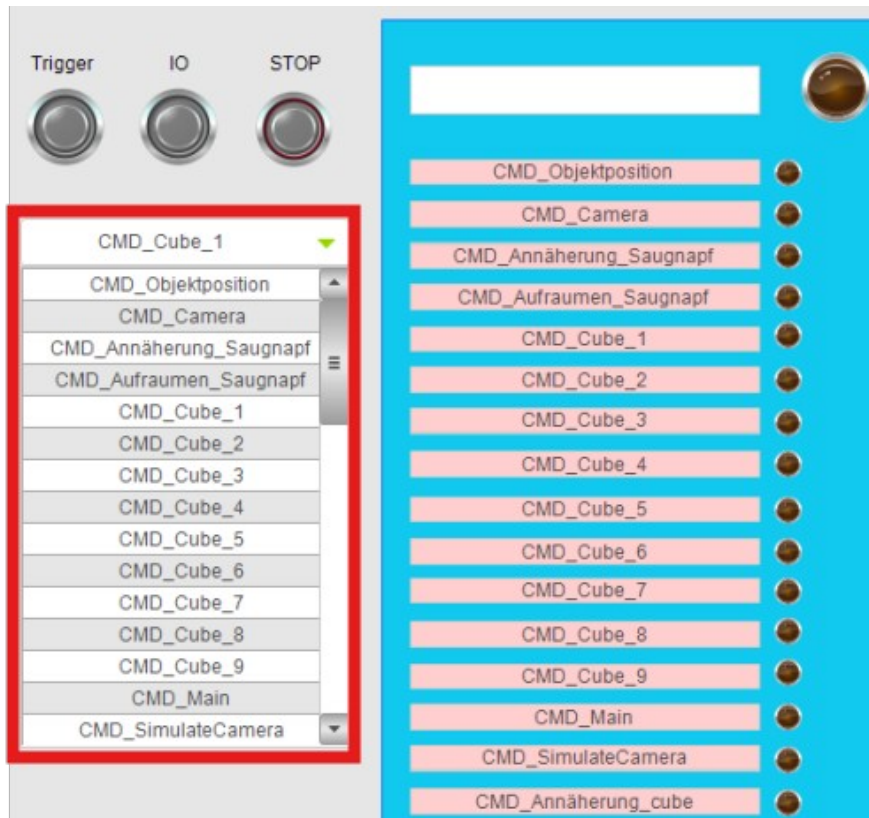


If the Python script is started before the CODESYS runtime, the OPC UA connection fails.

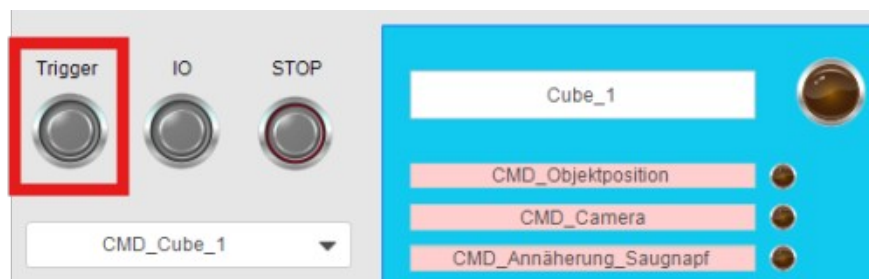
In that case, it is enough to stop the Python script and start it again.

## Starting a program from the interface

1. Select a program in the drop-down list



2. Press the **Trigger** button to confirm



3. Press the **IO** button to start the program



If another program is selected after pressing "Trigger", the **Stop** button must be pressed first.

Otherwise, the newly selected program will not be started.